Numerical Linear Algebra Toolkit

Project 1: Report on the Numerical Experiments

Aman Soni (team leader), Aditya Kumar, Aparna Singh, Anvesh Devulapalli

1. Introduction and package overview

This project delivers a small, tested Python package for solving linear systems **Ax = b** and for studying how accurate and reliable the resulting numerical solutions are. The core algorithms are implemented from scratch; NumPy is used only for basic array operations and as an independent reference for validation (np.linalg.solve, np.linalg.cond and matrix norms). The package has three parts:

- **Gaussian elimination with partial pivoting.** At each step the row with the largest entry in the current column is swapped into the pivot position, entries below the pivot are eliminated, and the triangular system is solved by back substitution. A pivot smaller than 10^{-14} in magnitude is reported as a singular matrix (ValueError). Cost: $O(n^3)$ for elimination, $O(n^2)$ for substitution.
- **LU factorization with partial pivoting.** The factors satisfy PA = LU, and a system is solved through Ly = Pb and Ux = y. The factorization costs $O(n^3)$ once; each further right-hand side then costs only $O(n^2)$, so a factorization can be reused.
- **Numerical diagnostics.** For a computed solution $\hat{x}$, the residual is $r = b - A\hat{x}$, the relative residual is $\|r\|_2/\|b\|_2$, and the normwise relative backward error is $\eta = \|r\|_2/(\|A\|_2\|\hat{x}\|_2 + \|b\|_2)$. The conditioning of the matrix is measured by $\kappa_2(A) = \|A\|_2\|A^{-1}\|_2$. All norms are 2-norms.

The package is object-oriented. Every solver implements one common interface, so the experiments can treat them interchangeably:

Class	Role
LinearSolver	Abstract base class: every solver provides solve(A, b) and can be called like a function.
GaussianEliminationSolver	Gaussian elimination with partial pivoting plus back substitution.
LUDecomposition	Stores P, L, U; solves with forward and back substitution; reusable for many right-hand sides.
LUSolver	LinearSolver that factorizes A and then solves.
MatrixDiagnostics	Properties of the matrix alone, such as the condition number.
SolutionDiagnostics	Inherits MatrixDiagnostics; residual, relative residual and backward error of a computed solution.

Table 1. Main classes of the numerical_linear_alg package.

2. Verification and workflow

Automated tests. The package ships with 66 pytest tests, all passing. They cover known solutions, row swaps and a tiny first pivot; singular, zero, non-square, mismatched and non-finite inputs; immutability of the caller's data and list inputs; agreement with NumPy on random systems; reconstruction PA = LU and the triangular structure of the factors; reuse of one factorization for many right-hand sides; the diagnostic formulas including the b = 0 edge case; and the experiment utilities.

Built-in cross-checks. Every experiment recomputes the residual, relative residual and backward error independently with NumPy and asserts agreement with the package diagnostics to a relative tolerance of 10^{-12} (reported as "Diagnostic validation: PASSED"). It also checks the package condition number against np.linalg.cond and reports each solver's distance from the NumPy solution. All validations passed in every experiment below.

Team workflow. Work was tracked on GitHub and merged into main only through pull requests: 13 merged pull requests, each approved by a teammate other than its author (see CONTRIBUTIONS.md). **Reproducibility.** From a clean environment, pip install -r requirements.txt, python -m pytest and python -m experiments.well_conditioned,

experiments.pivoting and experiments.hilbert regenerate every number in this report. The last digits can differ slightly between machines because of the underlying BLAS/LAPACK library.

3. Experiment 1: a well-conditioned system

We solve $Ax = b$ with $A = [[4, 1, 1], [1, 3, 0], [1, 0, 2]]$, exact solution $x = (1, 2, 3)^T$ and $b = Ax = (9, 7, 7)^T$. The condition number is $\kappa_2(A) = 3.32$, so rounding errors can be amplified by at most a small factor.

Solver	Solution error	Relative residual	Backward error
Gaussian elimination (partial pivoting)	0	0	0
LU factorization (partial pivoting)	0	0	0
NumPy	0	0	0

Table 2. Experiment 1: all three solvers reproduce the exact solution.

Both of our implementations recover x exactly in double precision, and the distance to the NumPy solution is 0. Zero error is what a small, well-conditioned integer system can produce; an error of order 10^{-16} would be equally consistent with machine precision. The experiment is the baseline for the next two: with a well-conditioned matrix and a stable algorithm, there is nothing to amplify rounding errors.

4. Experiment 2: why partial pivoting matters

Here $A = [[10^{-12}, 1], [1, 1]]$, $x = (1, 1)^T$ and $b = Ax$. The matrix is well conditioned ($\kappa_2 = 2.618$, the square of the golden ratio), so any large error must come from the algorithm and not from the problem. The first pivot is only 10^{-12}. Without pivoting the elimination multiplier is $1/10^{-12} = 10^{12}$; with partial pivoting the rows are swapped first and the multiplier is 10^{-12}.

Method	Computed solution	Solution error	Relative residual	Backward error
Gaussian elimination, partial pivoting	(1 − 2.2e-16, 1 + 2.2e-16)	3.14e-16	9.93e-17	4.91e-17
LU factorization, partial pivoting	identical to the above	3.14e-16	9.93e-17	4.91e-17
NumPy	identical to the above	3.14e-16	9.93e-17	4.91e-17
Gaussian elimination, no pivoting	(0.99986686, 1.0)	1.33e-04	5.95e-05	2.94e-05

Table 3. Experiment 2: the same matrix solved with and without pivoting.

The failure without pivoting can be followed step by step:

- Row 2 is replaced by row 2 − 10^{12}·row 1. This gives $a_{22}' = 1 - 10^{12} = -999\,999\,999\,999$ and, in floating point, $b_2' = 2 - 10^{12} \cdot b_1 = -999\,999\,999\,999.0001$. The original entries of the second row are swamped by the 10^{12} terms, and $b_1 = 1 + 10^{-12}$, stored with a rounding error of about 10^{-16}, is magnified to an absolute error of order 10^{-4}, which is also the spacing of doubles near 10^{12}.
- Back substitution gives $x_2 = b_2'/a_{22}' = 1 + 2.2\times10^{-16}$, which is accurate to one rounding error.
- Then $x_1 = (b_1 - x_2)/10^{-12}$. The numerator should be 10^{-12} but is computed as 9.9987×10^{-13}: the subtraction cancels almost every digit. Dividing by the tiny pivot multiplies the absolute error 1.3×10^{-16} by 10^{12}, so $x_1 = 0.99986686$ with an error of 1.3×10^{-4}.

The result is not a conditioning effect: $\kappa_2 = 2.6$ could amplify errors by a factor of a few at most, yet the error is 4×10^{11} times larger than with pivoting (1.33×10^{-4} against 3.14×10^{-16}). The backward error confirms that the algorithm itself is unstable: 2.9×10^{-5} without pivoting, 4.9×10^{-17} with it. Unlike the Hilbert experiment below, the residual here does expose the problem (relative residual 6×10^{-5}). With partial pivoting every multiplier is at most 1 in magnitude, so intermediate quantities stay of order 1, no catastrophic cancellation occurs and the three pivoted solvers agree with each other and with NumPy to the last digit. Pivoting is a remedy for instability, not for ill-conditioning.

5. Experiment 3: Hilbert matrices

We use $H_{ij} = 1/(i + j - 1)$ for n = 3, 5, 8, 10, with $x = (1, \ldots, 1)^T$ and $b = Hx$. These matrices are notoriously ill conditioned: $\kappa_2(H)$ grows from 5.2×10^2 at n = 3 to 1.6×10^{13} at n = 10. Table 4 and Figure 1 show the relative solution error $\|\hat{x} - x\|_2/\|x\|_2$ and the relative residual for each solver.

n	$\kappa_2(H)$	Relative solution error			Relative residual		
		Gauss	LU	NumPy	Gauss	LU	NumPy
3	5.24e+02	8.03e-15	6.31e-16	8.02e-15	1.09e-16	0	0
5	4.77e+05	3.49e-13	3.71e-13	1.68e-12	1.66e-16	1.62e-16	8.67e-17
8	1.53e+10	3.59e-08	1.20e-07	6.12e-08	2.68e-17	2.68e-17	6.56e-17
10	1.60e+13	8.24e-05	2.68e-04	8.67e-05	3.33e-17	1.11e-16	1.20e-16

Table 4. Experiment 3: solution error and residual for the three solvers (Gauss = Gaussian elimination with partial pivoting).

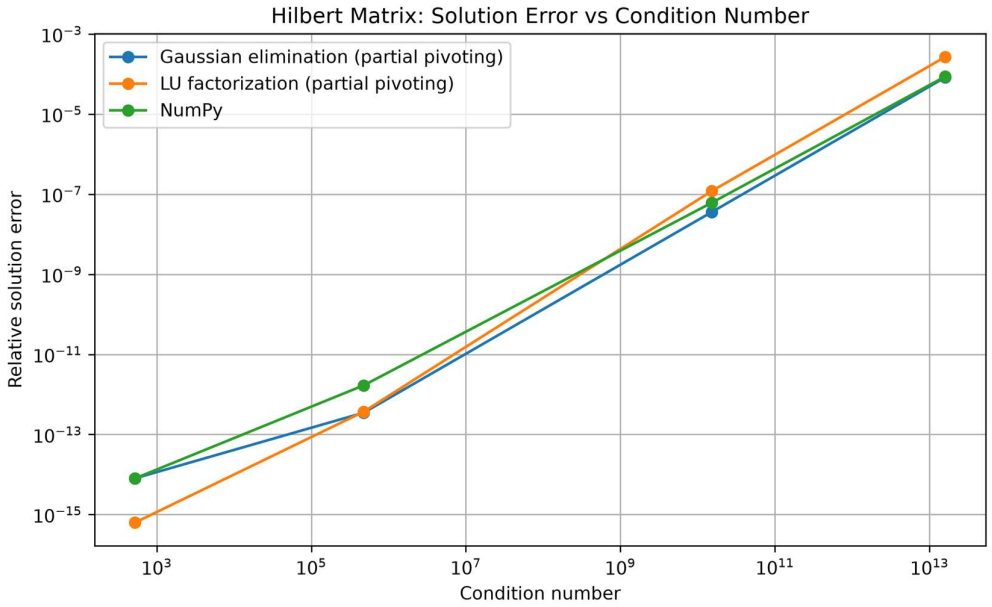


Figure 1. Relative solution error against condition number for the Hilbert matrices (log-log scale).

Accuracy is limited by the condition number

The error grows by ten orders of magnitude, from 8×10^{-15} at n = 3 to 8×10^{-5} at n = 10, while κ_2 grows by a factor of 3×10^{10}. In the log-log plot the Gaussian elimination points lie on a line of slope 0.95, that is, error ≈ κ_2. A rule of thumb says that about $\log_{10}\kappa_2$ digits are lost: with 16 digits available and $\kappa_2 = 1.6\times10^{13}$, only three to four correct digits remain, which matches the observed error of about 10^{-4}. Part of this loss is unavoidable for every solver: the vector $b = Hx$ is itself rounded to relative accuracy 10^{-16}, and an ill-conditioned matrix can amplify such a perturbation by up to κ_2.

A small residual does not imply a small error

For every size and every solver the relative residual stays at or below 1.7×10^{-16} and the backward error at or below 7.8×10^{-17}, i.e. at the level of the rounding unit. The solvers are therefore backward stable: each computed $\hat{x}$ is the exact solution of a problem within about 10^{-16} of the original. Yet the solution error at n = 10 is 8.2×10^{-5} for Gaussian elimination, twelve orders of magnitude above its residual of 3.3×10^{-17}. At n = 3 the computed residual of NumPy and of our LU solver is exactly 0, while their solution errors are 8.0×10^{-15} and 6.3×10^{-16}.

The reason is the relation $\hat{x} - x = A^{-1}r$, which gives the rigorous bound $\|\hat{x} - x\|_2/\|x\|_2 \le \kappa_2(A)\cdot\|r\|_2/\|b\|_2$. The residual only says how well $\hat{x}$ satisfies the equations. When A is nearly singular, many quite different vectors satisfy them almost equally well, so a tiny residual cannot certify an accurate solution. Table 5 confirms the bound for Gaussian elimination; it also held in the other six cases with a non-zero residual (LU and NumPy, n = 5, 8, 10).

n	$\kappa_2(H)$	Relative residual	$\kappa_2 \times$ relative residual	Actual relative error
3	5.24e+02	1.09e-16	5.73e-14	8.03e-15
5	4.77e+05	1.66e-16	7.91e-11	3.49e-13
8	1.53e+10	2.68e-17	4.09e-07	3.59e-08
10	1.60e+13	3.33e-17	5.34e-04	8.24e-05

Table 5. Gaussian elimination: the error never exceeds κ_2 times the relative residual.

Comparing the solvers

At each size the three solvers reach errors of the same order, and no solver is consistently more accurate. LU's error is about 3 times that of Gaussian elimination at n = 8 and 10, about equal at n = 5 and about 13 times smaller at n = 3. Mathematically both methods perform the same elimination, so the differences come from rounding in how the right-hand side is processed (updated during elimination versus forward substitution with stored multipliers); with one right-hand side and four sizes they should not be read as one algorithm being better. All of them are limited by the same conditioning.

6. Conclusions

- Both implementations work: on a well-conditioned system they reproduce the exact solution, they agree with NumPy in all experiments, and the 66 tests (including the PA = LU reconstruction) pass.
- Partial pivoting is essential for stability. On a well-conditioned 2×2 system, omitting it increased the error by a factor of 4×10^{11}, through a 10^{12} multiplier and cancellation.
- Accuracy is limited by conditioning: error $\approx \kappa_2 \times$ rounding unit. For the 10×10 Hilbert matrix only three to four digits are correct, whichever solver is used.
- Residuals and backward errors measure the stability of the algorithm, not the accuracy of the solution. A residual should always be reported together with the condition number, because $\kappa_2 \times$ residual bounds the error.
- The LU factorization object can be reused, which makes repeated solves with the same matrix cost $O(n^2)$ instead of $O(n^3)$.

Limitations and possible extensions

Each experiment uses one prescribed exact solution (and hence one right-hand side) and, for the Hilbert matrices, only four sizes, so the comparison between solvers is indicative rather than statistical. All results are in IEEE double precision, and their last digits depend on the machine's BLAS/LAPACK library. The package handles dense square systems and does not exploit sparsity or structure; the pivoting experiment contrasts partial pivoting only with no pivoting, not with complete pivoting. Natural extensions are iterative refinement, which can recover accuracy when the residual is computed in higher precision, and an estimate of the condition number that avoids forming A^{-1}.